

Serializable Isolation for Snapshot Databases

MICHAEL J. CAHILL, UWE RÖHM, and ALAN D. FEKETE
University of Sydney

20

Many popular database management systems implement a multiversion concurrency control algorithm called snapshot isolation rather than providing full serializability based on locking. There are well-known anomalies permitted by snapshot isolation that can lead to violations of data consistency by interleaving transactions that would maintain consistency if run serially. Until now, the only way to prevent these anomalies was to modify the applications by introducing explicit locking or artificial update conflicts, following careful analysis of conflicts between all pairs of transactions.

This article describes a modification to the concurrency control algorithm of a database management system that automatically detects and prevents snapshot isolation anomalies at runtime for arbitrary applications, thus providing serializable isolation. The new algorithm preserves the properties that make snapshot isolation attractive, including that readers do not block writers and vice versa. An implementation of the algorithm in a relational DBMS is described, along with a benchmark and performance study, showing that the throughput approaches that of snapshot isolation in most cases.

Categories and Subject Descriptors: H.2.4 [Database Management]: Systems—*Transaction processing*

General Terms: Algorithms, Performance, Reliability

Additional Key Words and Phrases: Multiversion concurrency control, serializability, snapshot isolation

ACM Reference Format:

Cahill, M. J., Röhm, U., and Fekete, A. D. 2009. Serializable isolation for snapshot databases. *ACM Trans. Datab. Syst.* 34, 4, Article 42 (December 2009), 42 pages.
DOI = 10.1145/1620585.1620587 <http://doi.acm.org/10.1145/1620585.1620587>

1. INTRODUCTION

Serializability is an important property when transactions are executed concurrently because it ensures that integrity constraints are maintained even if those constraints are not explicitly declared to the DBMS. If a DBMS enforces that all executions are serializable, then developers do not need to worry that inconsistencies in the data might appear as artifacts of concurrency or failure.

M. J. Cahill is now affiliated with Oracle Corporation.

Authors' address: email: {mjc,roehm,fekete}@it.usyd.edu.au.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies show this notice on the first page or initial screen of a display along with the full citation. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, to republish, to post on servers, to redistribute to lists, or to use any component of this work in other works requires prior specific permission and/or a fee. Permissions may be requested from Publications Dept., ACM, Inc., 2 Penn Plaza, Suite 701, New York, NY 10121-0701 USA, fax +1 (212) 869-0481, or permissions@acm.org.
© 2009 ACM 0362-5915/2009/12-ART20 \$10.00

DOI 10.1145/1620585.1620587 <http://doi.acm.org/10.1145/1620585.1620587>

It is well-known how to use strict two-phase locking (and various enhancements such as escrow locking and multigranularity locking) to control concurrency so that serializable executions are produced [Gray and Reuter 1993]. Some other concurrency control algorithms are known that ensure serializable execution, but these have not been widely adopted in practice, because they usually perform worse than a well-engineered implementation of strict two-phase locking (S2PL).

Snapshot isolation (SI) [Berenson et al. 1995] is an alternative approach to concurrency control, taking advantage of multiple versions of each data item. In SI, a transaction T sees the database state as produced by all the transactions that have committed before T starts, but no effects are seen from transactions that overlap with T . This means that SI never suffers from inconsistent reads. In a DBMS using SI for concurrency control, reads are never delayed because of concurrent transactions' writes, nor do reads cause delays in a writing transaction. In order to prevent Lost Update anomalies, SI does abort a transaction T when a concurrent transaction commits a modification to an item that T wishes to update. This is called the “First-Committer-Wins” rule.

Despite the attractive properties of SI, it has been known since SI was formalized in Berenson et al. [1995] that SI permits non-serializable executions. In particular, it is possible for a DBMS using SI to interleave some transactions, where each transaction preserves an integrity constraint when run alone, but where the final state after the interleaved execution does not satisfy the constraint. This occurs when concurrent transactions modify different items that are related by a constraint, and it is called the “Write Skew” anomaly.

Example 1.1. Suppose that a table Duties(DoctorId, Shift, Status) represents the status (“on duty” or “reserve”) for each doctor during each work shift. An undeclared invariant is that, in every shift, there must be at least one doctor on duty. A parametrized application program that changes a doctor D on shift S to “reserve” status, can be written as follows:

```
BEGIN TRANSACTION

UPDATE Duties SET Status = 'reserve'
WHERE DoctorId = :D
      AND Shift = :S
      AND Status = 'on duty'

SELECT COUNT(DISTINCT DoctorId) INTO tmp
FROM Duties
WHERE Shift = :S
      AND Status = 'on duty'

IF (tmp = 0) THEN ROLLBACK ELSE COMMIT
```

This program is consistent, that is, it takes the database from a state where the integrity constraint holds to another state where the integrity

constraint holds. However, suppose there are exactly two doctors $D1$ and $D2$ who are on duty in shift S . If we run two concurrent transactions that run this program for parameters $(D1, S)$ and $(D2, S)$ respectively, we see that using SI will allow both to commit (as each will see the other doctor's status for shift S as still unchanged, at "on duty"). However, the final database state has no doctor on duty in shift S , violating the integrity constraint.

Notice that the integrity constraint is violated even though the transaction checks it explicitly before committing, because the constraint is checked on the snapshot of data visible to the transaction rather than the final state resulting from interleaved execution.

Despite the possibility of corrupting the state of the database, SI has become popular with DBMS vendors. It often gives much higher throughput than strict two-phase locking, especially in read-heavy workloads, and it also provides users with transaction semantics that are easy to understand. Many popular and commercially important database engines provide SI, and some in fact use SI when serializable isolation is requested [Jacobs et al. 1995].

Because SI permits data corruption, and is so common, there has been a body of work on how to ensure serializable executions when running with SI as a concurrency control. The main techniques proposed so far [Fekete et al. 2005; Fekete 2005; Jorwekar et al. 2007] depend on doing a design-time static analysis of the application code, and then modifying the application if necessary in order to avoid the SI anomalies. For example, Fekete et al. [2005] shows how one can introduce write-write conflicts into the application, so that all executions will be serializable even when run with SI.

Using static analysis to make applications serializable when run at SI has a number of limitations. It relies on an education campaign so that application developers are aware of SI anomalies, and it is unable to cope with ad hoc transactions. In addition, this must be a continual activity as an application evolves: the analysis requires the global graph of transaction conflicts, so every minor change in the application requires renewed analysis, and perhaps additional changes (even in programs that were not altered). In this article, we instead focus on guaranteeing serializability for every execution of arbitrary transactions, while still having the attractive properties of SI, in particular much better performance than is allowed by strict two-phase locking.

1.1 Contributions

We describe a new concurrency control algorithm, called Serializable Snapshot Isolation (Serializable SI), with the following innovative combination of properties:

- The concurrency control algorithm ensures that every execution is serializable, no matter what application programs run.
- The algorithm never causes a read operation to block; nor do reads block concurrent writes.

- Under a range of conditions, the overall throughput is close to that allowed by SI, and better than that of strict two-phase row-level locking.
- The algorithm is easily implemented by small modifications to a data management platform that provides SI.

The key idea of our algorithm is to detect, at runtime, distinctive conflict patterns that must occur in every non-serializable execution under SI, and abort one of the transactions involved. This is similar to the way serialization graph testing (SGT) works, however our algorithm does not operate purely as a certification at commit-time, but rather aborts transactions as soon as the problem is discovered; also our test does not require any cycle-tracing in a graph, but can be performed by considering conflicts between pairs of transactions, and a small amount of information, which is kept for each of them. Our algorithm is also similar to optimistic concurrency control [Kung and Robinson 1981] but differs in that it only aborts a transaction when a pair of consecutive conflict edges are found, which is characteristic of SI anomalies. This leads to significantly fewer aborts than optimistic techniques that abort when any single conflict edge is detected. Our detection is conservative, so it does prevent every non-serializable execution, but it may sometimes abort transactions unnecessarily.

We describe prototype implementations of the algorithm in two open source database management systems: Oracle Berkeley DB [Olson et al. 1999] and the InnoDB transactional storage engine for MySQL [MySQL AB 2006], and we evaluate the performance of our implementation of Serializable SI compared to each product's implementations of Strict Two-Phase Locking (S2PL), Berkeley DB's implementation of SI, and a straightforward implementation of Snapshot Isolation that we added to InnoDB.

The remainder of the article is structured as follows: in Section 2 we give an introduction to snapshot isolation and current approaches to ensuring serializable isolation with SI; in Section 3 we describe the new Serializable SI algorithm; in Section 4 we describe the implementation in InnoDB and in Section 5 we evaluate its performance. Section 6 concludes.

2. BACKGROUND

2.1 Concurrency Control

There is a long history of related work on algorithms for multiversion concurrency control, much of it predating snapshot isolation. Reed [1978, 1983] introduces the idea that multiple versions of each data item can be used to provide read-only access without locking and describe a multiversion concurrent control algorithm based on timestamps for implementing atomic actions (closely related to transactions). However, this algorithm may abort read-only transactions if they try to read old versions of data items that are no longer available in the system, and require information when an update begins in order to predetermine a serialization order for updates.

Chan et al. [1983] describe the concurrency control algorithm in the CCA database management system that provides a consistent snapshot to each

read-only transaction and eliminates conflicts between read-only and update transactions. Updates use traditional two-phase locking (2PL) to guarantee serializable execution. Carey and Muhanna [1986] detail a simulation study of various multiversion concurrency control algorithms and show that all offer significant performance advantages over 2PL despite the overhead of maintaining versions and the possibility of additional disk accesses. Weihl [1987] describes a family of algorithms for multiversion concurrency control with particular focus on maintenance and cleanup of old versions of data items. Bober and Carey [1992] extend the CCA algorithm to row-level locking and describe some implementation techniques to improve performance. Mohan et al. [1992] apply multiversion concurrency control (called *transient versioning*) to a DBMS with row-level locking and indices.

In all of the algorithms described after Chan et al. [1983], read-only transactions are separated from updates and use different techniques to ensure serializable execution. In contrast, snapshot isolation provides uniform semantics for both read-only and update transactions and can thus be the primary concurrency control algorithm for a DBMS. With snapshot isolation, locking is entirely avoided for read operations regardless of whether they occur as part of an update transaction, no ANSI SQL anomalies are permitted and implementations have demonstrated very good performance. We focus on snapshot isolation because it is the technique implemented by a wide range of popular systems.

Serialization Graph Testing (SGT) schedulers maintain a graph representing the conflicts between all concurrent transactions in a system, and prevent operations that would create a cycle in the graph. As noted in Weikum and Vossen [2002], SGT schedulers are theoretically attractive but not practical to implement based on textbook descriptions, since they require space and time proportional to the square of the largest set of overlapping transactions. This is larger than the set of active transactions: if there is a long-running transaction, nodes must be added to the graph for all transactions that are concurrent with any part of the long-running transaction.

2.2 Snapshot Isolation

SI is a concurrency control approach that uses multiple versions of data to provide non-blocking reads. When a transaction T starts executing, it gets a conceptual timestamp $\text{start-time}(T)$; whenever T reads a data item x , it does not necessarily see the latest value written to x ; instead T sees the version of x that was produced by the last to commit among the transactions that committed before T started and also modified x (there is one exception to this: if T has itself modified x , it sees its own version). Thus, T appears to execute against a snapshot of the database, that contains the last committed version of each item at the time when T starts.

SI also enforces an additional restriction on execution, called the “First-Committer-Wins” (FCW) rule: two concurrent transactions that both modify the same data item cannot both commit. In practice, most implementations of SI use locking during updates to prevent a transaction from modifying an item if a

concurrent transaction has already modified it. The first transaction to acquire the lock for an item is permitted to update the item: if that transaction goes on to commit, any blocked transactions abort after the lock is released. This is sometimes referred to as “First-Updater-Wins,” but the effect on permitted transaction histories is the same as the more abstract First-Committer-Wins rule.

SI was introduced in the research literature in Berenson et al. [1995], and it has been implemented by the Oracle RDBMS, PostgreSQL, SQL Server 2005, and Oracle Berkeley DB. It provides significant performance improvements over serializability implemented with two-phase locking (S2PL) and it avoids many of the well-known isolation anomalies such as Lost Update or Inconsistent Read. In some systems that do not implement S2PL, including the Oracle RDBMS and PostgreSQL, SI is provided when serializable isolation is requested.

2.3 Write Skew

As noted in Berenson et al. [1995], SI does not guarantee that all executions will be serializable, and it can allow corruption of the data through interleaving between concurrent transactions that individually preserve the consistency of the data.

Example 2.1. Consider the interleaving of two transactions, T_1 and T_2 , withdrawing money from bank accounts x and y , respectively, subject to the constraint that $x + y > 0$. Here is an execution that can occur under SI:

$$r_1(x = 50)r_1(y = 50)r_2(x = 50)r_2(y = 50)w_1(x = -20)w_2(y = -30)c_1c_2.$$

Each of the transactions begins when the accounts each contain \$50, and each transaction in isolation maintains the constraint that $x + y > 0$: when T_1 commits, it calculates $x + y = -20 + 50 = 30$, and when T_2 commits, it calculates $x + y = 50 + -30 = 20$. However, the interleaving results in $x + y = -50$, so the constraint has been violated. This type of anomaly is called a *write skew*.

We can understand these situations using a multiversion serialization graph (MVSG). There are a number of definitions of this in the literature, because the general case is made complicated by uncertainty over the order of versions (which indeed renders it NP-Hard to check for serializability of a multiversion schedule). For example, there are definitions in Bernstein and Goodman [1983], Hadzilacos [1988], Raz [1993], and Adya [1999].

With snapshot isolation, the definitions of the serialization graph become much simpler, as versions of an item x are ordered according to the temporal sequence of the transactions that created those versions (note that First-Committer-Wins ensures that among two transactions that produce versions of x , one will commit before the other starts). In the MVSG, we put an edge from one committed transaction, T_1 , to another committed transaction, T_2 , in the following situations: T_1 produces a version of x , and T_2 produces a later version of x (this is a *ww*-dependency); T_1 produces a version of x , and T_2 reads this (or a later) version of x (this is a *wr*-dependency); T_1 reads a version of x , and T_2

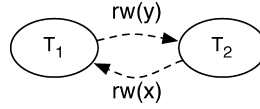


Fig. 1. Serialization graph for transactions exhibiting write skew.

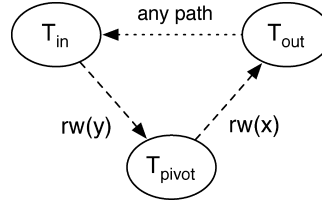


Fig. 2. Generalized dangerous structure in the MVSG.

produces a later version of x (this is a rw -dependency). In Figure 1 we show the MVSG for the history with write skew, discussed above. In drawing our MVSG, we will follow the notation introduced in Adya [1999], and use a dashed edge to indicate an rw -dependency.

As usual in transaction theory, the absence of a cycle in the MVSG proves that the history is serializable. Thus it becomes important to understand what sorts of MVSG can occur in histories of a system using SI for concurrency control. Adya [1999] showed that any cycle produced by SI has two rw -dependency edges. This was extended in Fekete et al. [2005], which showed that any cycle must have two rw -dependency edges that occur consecutively, and further, each of these edges is between a pair of transactions that are concurrent with each other.

We adopt some terminology from Fekete et al. [2005], and call an rw -dependency between concurrent transactions a *vulnerable edge*; we call the situation where two consecutive vulnerable edges occur in a cycle as a *dangerous structure*. It is illustrated in Figure 2. We refer to the transaction at the junction of the two consecutive vulnerable edges as a *pivot* transaction, T_{pivot} , in the figure. We call the transaction that is the source of the edge into the pivot the *incoming* transaction (T_{in}), and the remaining transaction the *outgoing* transaction, T_{out} .

The theory of Fekete et al. [2005] shows that in any non-serializable execution allowed by SI there is a pivot transaction and the outgoing transaction commits first of all transactions in the cycle.

Example 2.2. We take an interesting example from Fekete et al. [2004] to illustrate how a dangerous structure may occur at runtime. Consider the following three transactions:

$$\begin{aligned} T_{pivot}: & \text{r}(y) \text{ w}(x) \\ T_{out}: & \text{w}(y) \text{ w}(z) \\ T_{in}: & \text{r}(x) \text{ r}(z). \end{aligned}$$

These three transactions can interleave such that T_{in} , a read-only transaction, sees a state that could never have existed in the database had T_{pivot} and T_{out}

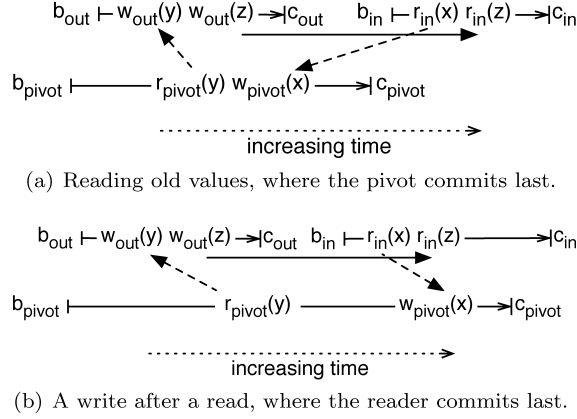


Fig. 3. Some SI anomalies at runtime.

executed serially. The issue is that if T_{pivot} and T_{out} run concurrently, then in a serial schedule, T_{pivot} would have to be serialized before T_{out} because T_{pivot} 's read of y will not see the update from T_{out} . However, the read-only transaction T_{in} can be interleaved such that it reads a snapshot containing updates committed by T_{out} but not T_{pivot} . If T_{in} is omitted, T_{pivot} and T_{out} are serializable because there is only a single antidependency from T_{pivot} to T_{out} .

Two of the possible non-serializable interleavings of these three transactions are illustrated in Figure 3. These diagrams should be read from left to right; the arrows indicate the rw -dependencies between transactions. In Figure 3(a), both reads occur after the writes. In Figure 3(b), T_{in} reads x before it is written by T_{pivot} .

Also note in Figure 3(a) that $r_{in}(x)$ occurs after c_{pivot} . That is, the read-write conflict on data item x is not detected until after the pivot has committed. One of the challenges that an algorithm must overcome to detect SI anomalies at runtime is that we cannot always know when a transaction commits whether it will have consecutive vulnerable edges.

We allow for the possibility of *blind writes* in our formalism where there is no read of a data item before it is written, such as the $w(x)$ in T_{pivot} of the example above. This increases the generality of the approach without introducing complexity into either the discussion or the algorithm.

2.4 Phantoms

Throughout the discussion so far, we have followed typical concurrency control theory and assumed that a transaction is a sequence of reads and writes on named data items. In general, a relational database engine must also deal with predicate operations (such as SQL “where” clauses). These mean that a concurrency control algorithm must also consider phantoms, where an item created or deleted in one transaction would change the result of a predicate operation in a concurrent transaction if the two transactions executed serially.

The problem was identified in Eswaran et al. [1976], but the general-purpose “predicate locking” solution suggested there has not been widely adopted because of the difficulty in testing mutual satisfiability of predicates. Instead, locking DBMS implementations commonly use algorithms based on “next-key locking.” In these, a range of key space is protected against concurrent insertion or deletion by acquiring a shared lock on the next row in order, as a scan is made to check whether rows match a predicate. The scan might be through the data records or through an index. Inserts and deletes follow the same protocol, obtaining an exclusive lock on the row after the one being inserted or deleted. The result of this locking protocol is that a range scan prevents concurrent inserts or deletes within the range of the scan, and vice versa. Highly optimized versions of this approach have been proposed by Mohan [1990], Mohan and Levine [1992], and Lomet [1993].

InnoDB uses a modification of next-key locking involving “gap” locks [MySQL AB 2006]. The motivation for “gap” locks is that a read of the range $[a, c)$ should not conflict with $\text{update}(c)$. Similarly, $\text{insert}(b)$ should not prevent a concurrent $\text{update}(c)$. A gap lock on x is conceptually a lock on the gap just before x : it conflicts with other gap locks on x , but does not conflict with locks on item x itself. Logically, a gap lock is equivalent to using a different key in the lock table for the same data item x . Scans, inserts, and deletes acquire gap locks on the next key (or the special supremum key for the last key in a table) to prevent phantoms without introducing conflicts with ordinary reads and updates on the next key.

2.5 Related Work

An extensive line of research has considered, not the serializability of particular executions allowed by SI, but rather the question of whether a given set of application programs is guaranteed to generate serializable executions when run on a system with SI as the concurrency control mechanism. This problem was addressed in Fekete [1999] and the techniques were refined in Fekete et al. [2005]. The key to this work is to consider a static analysis of the possible conflicts between application programs. Thus a static dependency graph, or SDG, is drawn, with an edge from program P_1 to P_2 , if there can be an execution where P_1 generates a transaction T_1 , P_2 generates T_2 , and there is a dependency edge from T_1 to T_2 . It was shown how a dangerous structure in the MVSG can be related to a similar structure in the SDG, and this justified the intuition of experts, who had previously decided that every execution of the TPC-C benchmark [Transaction Processing Performance Council 2005] is serializable on a platform using SI. As well as showing how to prove that certain programs generate only serializable executions, Fekete et al. [2005] proposed that one could modify transaction programs so that they fall into this class. The modifications typically involve introducing extra write-write conflicts between transaction programs that might give rise to transactions joined by a vulnerable edge within a dangerous structure.

The theory of Fekete et al. [2005] was extended in Fekete [2005] to the case where some transactions use SI and others use S2PL (as is possible with

Microsoft SQL Server 2005 or Oracle Berkeley DB). Performance studies [Alomari et al. 2008] indicate that modifying applications to ensure serializability under SI can be done without significant cost when the appropriate technique is used.

Jorwekar et al. [2007] describe a system that automates the analysis of program conflicts using syntactic features of program texts, such as the names of the columns accessed in each statement. Working purely at the level of SQL syntax results in coarse granularity of conflict detection, resulting in a large proportion of false positives, where benign transaction programs were flagged by the tool for further (manual) analysis. When dangerous access patterns were manually confirmed, anomalies were avoided by modifying the transaction programs by hand. One important finding of their work is that snapshot isolation anomalies do exist in applications developed using tools and techniques that are common throughout the software industry.

A preliminary version of this work appears in Cahill et al. [2008], which added a new implementation of serializable isolation to Oracle Berkeley DB [Olson et al. 1999]. The present article extends the approach to a relational DBMS using row-level locking and versioning, and deals with the associated problem of phantoms. The evaluation we give here is based on TPC-C [Transaction Processing Performance Council 2005], which is more complex and representative of typical OLTP workloads than the SmallBank benchmark presented in Alomari et al. [2008] and used in Cahill et al. [2008].

An alternative approach to ensuring correctness when running on platforms with SI is in [Bernstein et al. 2000], where conditions are given to ensure all executions preserve given integrity constraints, without necessarily being serializable.

Others have previously suggested ways to alter multiversion concurrency control algorithms in a DBMS in order to avoid non-serializable executions at run-time. Proposals related to certification through serialization graph testing (SGT) are given in Tada et al. [1997], Shi and Perrizo [2002], and Yang [2007]. These suggestions have not focused on feasibility of implementation within a DBMS. In particular, the space required to represent complete conflict graphs and the overhead required to maintain them may be prohibitive. There is at least one commercial system purported to be based on SGT [Sarin 2009], but the details are proprietary.

3. SERIALIZABLE SNAPSHOT ISOLATION

The essence of our new concurrency control algorithm is to allow standard SI to operate, but to add some bookkeeping so we can dynamically detect cases where a non-serializable execution could occur, and then we abort one of the transactions involved. This makes the detection process a delicate balance: if there are any cases that are not detected, some non-serializable execution may emerge (counter to our goal of having true serializability guarantees for the applications), but if we detect too many cases, then performance will suffer, since unnecessary aborts waste resources. As a third factor in designing an algorithm, we also need to keep the overhead cost of detection low. One can imagine a

concurrency control algorithm that aborts a transaction exactly when an operation will result in a non-serializable execution; this would be a serialization graph testing (SGT) scheduler using the appropriate multiversion serialization graph. SGT schedulers, however, require expensive cycle detection calculations on each operation and would be very expensive. According to Weikum and Vossen [2002, page 170],

“Note that an SGT scheduler, although attractive from a theoretical point of view . . . is not practical to implement. For example, the (worst case) *space* required to maintain the serialization graph grows with the square of the number of relevant transactions, some of which are not even active anymore. We even need to keep around the information about the read sets and write sets of such already-committed transactions. In addition, and even more importantly, the testing of cycles in the graph, albeit only linear in the number of edges (i.e., quadratic in the number of nodes in the worst case), may be unacceptable at run time simply because such a test would be invoked very frequently. After all, concurrency control measures are part of a data server’s innermost loops, and their run-time overhead is thus an extremely critical issue.”

Thus we accept a small chance of unnecessary aborts, in order to keep the detection overhead low.

The key design decision in our new algorithm is thus the situations in which potential anomalies are detected. We use the theory of Adya [1999] and its extension from Fekete et al. [2005], where some distinctive conflict patterns are shown to appear in every non-serializable execution of SI. The building block for this theory is the notion of an *rw*-dependency (also called an “anti-dependency”), which occurs from transactions T_1 to T_2 if T_1 reads a version of an item x that is older than a version of x produced by T_2 . In Adya [1999] it was shown that in any non-serializable SI execution, there are two *rw*-dependency edges in a cycle in the multiversion serialization graph. Fekete et al. [2005] extended this to show that there are two *rw*-dependency edges which form consecutive edges in a cycle. Furthermore, each of these *rw*-edges involves two transactions that are active concurrently and the outgoing transaction from the pivot, T_{out} in Figure 2, must commit first.

Our Serializable SI concurrency control algorithm detects a potentially non-serializable execution whenever it finds two consecutive *rw*-dependency edges in the serialization graph, where each of the edges involves two transactions that are active concurrently. Whenever such a situation is detected, one of the transactions will be aborted. To support this algorithm, the DBMS maintains, for each transaction, two Boolean flags: `T.inConflict` indicates whether there is an *rw*-dependency from another concurrent transaction to T , and `T.outConflict` indicates whether there is an *rw*-dependency from T to another concurrent transaction. Thus a potential non-serializability is detected when `T.inConflict` and `T.outConflict` are both true.

We note that our algorithm is conservative: if a non-serializable execution occurs, there will be a transaction with `T.inConflict` and `T.outConflict`.

However, we do sometimes make false positive detections; for example, an unnecessary detection may happen because we do not check whether the two *rw*-dependency edges occur within a cycle. It is also worth mentioning that we do not always abort the particular pivot transaction T for which $T.inConflict$ and $T.outConflict$ is true; this is often chosen as the victim, but sometimes the victim is the transaction that has an *rw*-dependency edge to T , or the one that is reached by an edge from T .

It is possible with our Serializable SI algorithm that a transaction is restarted after being aborted following detection of an unsafe pattern of conflicts, and the restarted transaction again causes an unsafe pattern of conflicts and is forced to abort. The impact of this issue is ameliorated by two facts:

- the rate of detection of unsafe conflict patterns is very low in all the cases we have measured; and
- it is very rare for our algorithm to abort pure queries, and updates involved in conflicts usually wait for at least one write lock before they are aborted. By the time the write lock is granted, the conflicting operation has completed, so if the update is forced to retry, it will not have the same pattern of conflicts.

Our performance evaluation in Section 5 supports our belief that cascading aborts do not occur often enough in practice with Serializable SI to cause a measurable impact on throughput.

How can we keep track of situations where there is an *rw*-dependency between two concurrent transactions? There are two different ways in which we notice such a dependency. One situation arises when a transaction T reads a version of an item x , and the version that it reads (the one that was valid at T 's start time) is not the most recent version of x . In this case the writer U of any more recent version of x was active after T started, and so there is an *rw*-dependency from T to U . When we see this, we set the flags $T.outConflict$ and $U.inConflict$ (and we check for consecutive edges and abort a transaction if needed). This allows us to find *rw*-dependency edges for which the read occurs in real-time after the write that is logically later. However, it does not account for edges where the read occurs first, and at a later real-time, a new version is created by a concurrent transaction.

To notice these other *rw*-dependency cases, we rely on the lock management infrastructure of the DBMS. A normal exclusive lock is taken when a new version is created; most SI implementations do keep such exclusive locks anyway, as a way to enforce the First-Committer-Wins rule.

We also introduce a new lock mode called SIREAD, which is used to record the fact that an SI transaction has read a version of an item. However, obtaining the SIREAD lock does not cause any blocking, even if an exclusive lock is held already, and similarly an existing SIREAD lock does not delay granting of an exclusive lock; instead, the presence of both SIREAD and exclusive locks on an item is a sign of an *rw*-dependency, and so we set the appropriate *inConflict* and *outConflict* flags on the transactions that hold the locks. One difficulty, which we discuss later, is that we need to keep the SIREAD locks that T obtained, even after T is completed, until all transactions concurrent with T have completed.

modified begin(T):

```

1   existing SI code for begin(T)
2   set T.inConflict = T.outConflict = false
    
```

Fig. 4. Modifications for begin(T), called to start a Serializable SI transaction.

modified commit(T):

```

1   atomically:
2       set T.running = false
3       if T.inConflict and T.outConflict:
4           abort(T)
5           return UNSAFE_ERROR
6
7   existing SI code for commit(T), do not release SIREAD locks
8
9   if T holds any SIREAD locks:
10       suspend T, keeping SIREAD locks active
11   if T was the oldest active transaction:
12       clean up any suspended transactions older than active transactions
    
```

Fig. 5. Modifications to commit(T), called to commit a Serializable SI transaction, checking that no unsafe pattern of conflicts has been detected.

3.1 The Basic Algorithm

We present pseudocode for the Serializable SI concurrency control algorithm in Figures 4 to 7. For ease of exposition, we start here describing a simplified version of the algorithm and give an argument for its correctness, then in Section 3.4 we extend the basic algorithm to detect phantoms, and in Sections 3.5 and 3.6 we describe some optimizations.

In describing the algorithm, we make some simplifying assumptions.

- (1) For any data item x , we can efficiently get the list of locks held on x .
- (2) For any lock l , we can efficiently get $l.owner$, the transaction object that requested the lock.
- (3) For any version xt of a data item x , we can efficiently get $xt.creator$, the transaction object that created that version.
- (4) When finding a version of item x valid at some given timestamp, we can efficiently get the list of other versions of x that have later timestamps.
- (5) There is latching or other concurrency control in the DBMS so that read and write operations involving a data item x have an unambiguous order.

These assumptions are true for both of our prototype target systems (Berkeley DB and InnoDB). In particular, both lock tables are structured for efficient lookup of locks, there is a direct pointer from locks to the owning transaction, and there is a table lookup required to find the transaction that created a given version of a data item. We discuss in Section 4.5 how to implement the algorithm if these assumptions do not hold.

```

modified read(T, x):
1   get lock(key=x, owner=T, mode=SIREAD)
2   if there is a conflicting EXCLUSIVE lock(wl) on x:
3       set wl.owner.inConflict = true
4       set T.outConflict = true
5
6   existing SI code for read(T, x)
7
8   for each version (xNew) of x that is newer than what T read:
9       atomically:
10          if xNew.creator is committed and xNew.creator.outConflict:
11              abort(T)
12              return UNSAFE.ERROR
13          set xNew.creator.inConflict = true
14          set T.outConflict = true

```

Fig. 6. Modifications to read(T, x), called to read a data item in a Serializable SI transaction.

```

modified write(T, x, xNew):
1   get lock(key=x, owner=T, mode=EXCLUSIVE)
2
3   for each conflicting SIREAD lock(rl)
4       where rl.owner.running or commit(rl.owner) > begin(T):
5           atomically:
6               if rl.owner is committed and rl.owner.inConflict:
7                   abort(T)
8                   return UNSAFE.ERROR
9               set rl.owner.outConflict = true
10              set T.inConflict = true
11
12   existing SI code for write(T, x, xNew)
13   do not get the EXCLUSIVE lock again

```

Fig. 7. Modifications to write(T, x, xNew), called to modify a data item in a Serializable SI transaction.

The data structure changes required by the algorithm are the following.

- two Boolean flags are added to each transaction object: T.inConflict indicates whether or not there is an *rw*-dependency from a concurrent transaction to *T*, and T.outConflict indicates whether there is an *rw*-dependency from *T* to a concurrent transaction; and
- the new SIREAD mode for locks, assuming a lock manager that already keeps standard exclusive locks.

The Serializable SI algorithm modifies the operations of standard snapshot isolation as shown in Figures 4 to 7. In each case, the processing includes the usual processing of the operation by the SI protocol plus some extra steps.

The modification to the code to begin a transaction is given in Figure 4. This establishes the two new Boolean flags used by the Serializable SI algorithm, both initially false.

The modification to transaction commit is given in Figure 5. First, the modified code must mark that the transaction is no longer running. The definition of whether a transaction “is running” deserves some discussion. The intent is to ensure that the conflict bits will cause a running transaction to abort, or else if the transaction would have aborted but has already gone on to commit, the conflicting transaction must abort instead. So this test for whether a transaction is running must be atomic with respect to the check of the conflict flags at the beginning of `commit(T)`. After that check, a transaction can no longer abort due to the conflict flags being set, and should no longer be considered running.

Then in line 3 of Figure 5, we check the two new flags we have added to the transaction handle. If both flags are set, that indicates that both an incoming and outgoing conflict were detected while the transaction was running, so it may be the *pivot* of a cycle in the conflict graph. If two consecutive edges are detected here, the transaction is aborted rather than continuing with the commit, and an error is returned to the application to indicate that an error was detected.

If no unsafe pattern of conflicts is detected, we continue with the existing code to commit the transaction. Then if the transaction holds any SIREAD locks, it is temporarily suspended until all concurrent transactions have completed. In addition, we check whether the committing transaction was the oldest active transaction in the system, and if so, clean up any suspended transactions that could no longer participate in a conflict. This is discussed in more detail in Section 3.2.

When acquiring locks in Figures 6 and 7, the notation `key=x` indicates that a lock is acquired on the data item `x`, not any particular version or value that `x` holds. Locks acquired with `owner=T` are acquired on behalf of transaction `T`, and do not conflict with other locks held by `T`.

3.2 Transaction Lifecycle Changes

For the Serializable SI algorithm, it is important that the engine have access to information about a transaction `T`—its transaction record, including `T.inConflict` and `T.outConflict`, as well as any SIREAD locks it obtained—even after `T` has completed. The transaction record for `T` and any SIREAD locks acquired by `T` must be kept as long as there is any active transaction that overlaps with `T`. That is, we can only remove information about `T` after the end of every transaction that started before `T` completed. For this reason, transactions that commit holding an SIREAD lock are suspended temporarily rather than being immediately cleaned up in Figure 5. This suspend operation does not delay the client: the commit returns immediately, but the transaction record is kept so that future conflicts can be detected correctly.

Suspending transactions during commit implies that the suspended transactions are cleaned up at some later point. It is important for efficiency that the set of suspended transactions and the lock table do not grow unnecessarily large. In Section 4 we describe how this information is managed in the InnoDB implementation—in particular, how the space allocated to transaction objects is reclaimed.

3.3 Correctness

The Serializable SI algorithm ensures that every execution is serializable, and thus that data integrity is preserved (under the assumption that each transaction is individually coded to maintain integrity). Here we outline the argument that this is so. Note that here we only consider transactions consisting of read and write operations on individual data items. Predicate reads and the associated problem of phantoms are discussed in Section 3.4.

By Theorem 2.1 from Fekete et al. [2005], which shows that in any non-serializable execution there is a dangerous structure, we are done, provided that we can establish the following: whenever an execution contains a dangerous structure (transactions T_{in} , a pivot T_{pivot} , and T_{out} , such that there is an rw -dependency from T_{in} to T_{pivot} and T_{in} is concurrent with T_{pivot} , and also there is a rw -dependency from T_{pivot} to T_{out} and T_{pivot} is concurrent with T_{out}), then one of the transactions is aborted. In this situation, we must consider the possibility that $T_{in} = T_{out}$, which is the classic example of write skew.

Our algorithm has an invariant, that whenever the execution has an rw -dependency from T to U , and the transaction objects for both T and U exist, then both $T.outConflict$ and $U.inConflict$ are set to true. By definition, the rw -dependency comes from the existence of a read by T that sees some version of x , and a write by U that creates a version of x that is later in the version order than the version read by T .

One of these operations ($read(T, x)$ and $write(U, x)$) will happen first, because we assume that the database engine will perform some latching during their execution, and the other will happen later. The rw -dependency is present in the execution once the second of these operations occurs. If this second operation is $read(T, x)$, then at the time that operation is processed, there will already be the version of x created by U ; the pseudocode in Figure 6 shows that we explicitly set both flags as required. On the other hand, if the $write(U, x)$ occurs after $read(T, x)$, then at the time of the write, T will hold a SIREAD lock on x and the pseudocode in Figure 7 shows that both flags are set.

The modified $read(T, x)$ operation cannot omit the check for a conflicting exclusive lock or the check for newer versions of x . If either check were omitted, a race with a write operation on x would be possible, since we make no assumption about mutual exclusion between operations outside the of the lock subsystem, including obtaining a version of a particular item. If the read operation did not check for conflicting exclusive locks, the following interleaving of threads would be possible:

T get lock($x, T, SIREAD$) read(T, x)	U get lock($x, U, EXCLUSIVE$) write($U, x, xNew$).
---	--

The result is that neither thread detects the conflict.

Based on the invariant just described, we now must argue that one of the transactions in any dangerous structure is aborted. If both rw -dependencies

exist at the time T_{pivot} completes, then the code in Figure 5 will notice that both `inConflict` and `outConflict` are set for T_{pivot} (because of the invariant), and so T_{pivot} will be aborted when it requests to commit.

If, however, T_{in} reads a data item x written by T_{pivot} after T_{pivot} has committed, then a new version, x_{New} , will be detected at line 9 of Figure 6. If T_{pivot} committed without holding an SIREAD lock, it would not have been suspended, so it is possible that the creator of x_{New} is no longer available. However, in this case, T_{pivot} was a pure update, so it could never have a read-write conflict with a transaction, and thus could not be part of a cycle.

Assuming that T_{pivot} is the creator of x_{New} and is available at line 10 of Figure 6, if an outgoing conflict has already been detected for T_{pivot} , we must abort the reader, T_{in} , in order to prevent a cycle. Otherwise, conflicts are marked as usual in both transactions.

Similarly, if T_{out} writes a data item x that T_{pivot} read, then T_{pivot} must have taken an SIREAD lock on x and so would have been suspended during the commit. Then when T_{out} performs the write operation on x , the code in Figure 7 will find T_{pivot} 's SIREAD lock and detect the conflict if the two transactions were concurrent. If T_{pivot} already has an outgoing edge, we must abort T_{out} in order to prevent a cycle. Otherwise, the conflicts are marked on both transactions as before.

Therefore, even if both rw -dependencies occur after T_{pivot} commits, the algorithm will attempt to set both flags in T_{pivot} 's transaction structure, and the transaction responsible for the last detected dependency will be aborted.

In summary, the argument for correctness is as follows.

- (1) Nonserializable executions under SI consist of a cycle including two consecutive rw -dependencies.
- (2) Our algorithm detects every rw -dependency.
- (3) When two consecutive rw -dependencies are detected, at least one transaction is aborted, which breaks the cycle.

The exhaustive testing of the implementation that we describe in Section 4.6 further supports this argument for the algorithm's correctness.

3.4 Detecting Phantoms

A concurrency control algorithm for a relational DBMS must consider phantoms, where an item is created or deleted in one transaction, and the result is incorrectly not seen in a predicate operation in a concurrent transaction. Simple record-level locking, where readers simply lock the records they read and writers only lock the records they insert or delete, is not sufficient to prevent phantoms, because a record can be inserted that is logically within a range read by a concurrent query with no lock conflict occurring. Standard snapshot isolation also allows a form of write skew due to phantoms that must be prevented in order to provide serializable execution.

The approach to preventing phantoms commonly used in locking DBMS implementations including InnoDB is based on "next-key locking," including its "gap locking" refinement, as described in Section 2.4. In these schemes, a range

```

modified scanRead(T, x):
1   get gap lock(key=next(x), owner=T, mode=SIREAD)
2   if there is a conflicting EXCLUSIVE gap lock(wl) on next(x):
3       set wl.owner.inConflict = true
4       set T.outConflict = true
5
6   modified Serializable SI read(T, x)

```

Fig. 8. Modifications to scanRead(T, x), called when x is read during a scan to evaluate a predicate in a Serializable SI transaction.

```

modified insert(T, x) and delete(T, x):
1   get gap lock(key=next(x), owner=T, mode=EXCLUSIVE)
2
3   for each conflicting SIREAD gap lock(rl) on next(x)
4       where rl.owner.running or commit(rl.owner) > begin(T):
5           if rl.owner is committed and rl.owner.inConflict:
6               abort(T)
7               return UNSAFE_ERROR
8           set rl.owner.outConflict = true
9           set T.inConflict = true
10
11   insert: modified Serializable SI code for write(T, x, x)
12   delete: modified Serializable SI code for write(T, x, NULL)

```

Fig. 9. Algorithms for insert(T, x) and delete(T, x) in Serializable SI transactions.

of key space is protected against concurrent inserts or deletes by acquiring a shared lock on the gap after each row in order, as a scan is made, to check whether rows match a predicate. Inserts and deletes follow the same protocol, obtaining an exclusive lock on the gap after the row being inserted or deleted.

Our solution for detecting phantoms in snapshot isolation builds on gap locking by adding the SIREAD lock mode to gap locks for scans and using these locks to detect predicate read-write conflicts between transactions. In Figures 8 and 9, we provide an extension to the basic Serializable SI algorithm to prevent phantoms.

The notation $\text{next}(x)$ refers to the key in the table that sorts after x , or a special supremum key if x is the last key in the table.

If a predicate read operation occurs before the conflicting write operation, the enhanced Serializable SI algorithm given here detects the predicate-*rw*-conflict between transactions correctly.

In the case where a predicate read is interleaved after the conflicting write operation, the read would find a data item with a creation or deletion timestamp greater than the read timestamp of the predicate read. In other words, a row will be skipped because no version of the row was visible when the transaction performing the predicate read began or a row used by the predicate read has since been deleted. In either case, the newer versions of the row can be used to detect this kind of conflict as described in Figure 6 without further modification.

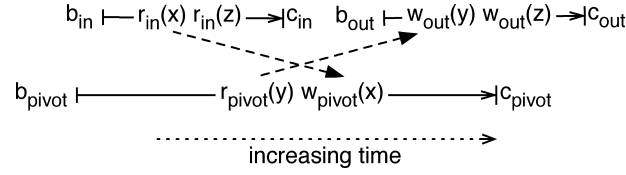


Fig. 10. False positive: no path from T_{out} to T_{in} .

This is implemented in our InnoDB prototype by building on the existing gap locking protocol built into InnoDB to prevent phantoms for the locking serializable isolation level. Insert and delete operations have already acquired EXCLUSIVE locks on the gap, so all that was necessary was to have SIREAD locks for predicate reads follow the same protocol, and catch when they conflict in exactly the same way as with ordinary row locks.

3.5 False Positives

Our algorithm uses a conservative approximation to cycle detection in the graph of transaction conflicts, and as such may cause some benign transactions to abort. The problem is that the two flags we have added to each transaction cannot indicate whether there is a complete cycle in the conflict graph. On the other hand, we expect that general cycle detection in the full conflict graph would require a prohibitive amount of additional memory and CPU time.

In particular, the interleaving of transactions in Figure 10 will set `inConflict` on T_{pivot} when it executes $w_{pivot}(x)$ and finds the SIREAD lock from T_{in} . Then `outConflict` will be set on T_{pivot} when T_{out} executes $w_{out}(y)$ and finds T_{pivot} 's SIREAD lock. During the commit of T_{pivot} , the two flags will be checked and since both are set, T_{pivot} will abort. However, this interleaving is equivalent to the serial history $\{T_{in}, T_{pivot}, T_{out}\}$ because T_{in} precedes T_{out} and hence there is no path of dependencies from T_{out} to T_{in} .

To address this, we have enhanced the algorithm by adding incoming and outgoing *transaction references* rather than single bits. These references are initially set to NULL, then set to refer to the transaction that first causes each conflict. If there are multiple conflicts of either type, the reference is set to point to the transaction itself. Setting the reference to either NULL or to the transaction itself takes the place of a single bit set to 0 or 1 in the basic algorithm.

The theory of Fekete et al. [2005] shows that in any nonserializable execution, there is a dangerous structure where the outgoing transaction, T_{out} in Figure 2 commits before all others in the cycle. Thus our algorithm still ensures serializability, even if we allow a pair of consecutive *rw*-dependencies, as long as we prevent situations where T_{out} commits first. Our algorithm does not track all transactions involved in the cycle, but if T_{out} commits before all transactions in the cycle, it must at least commit before either T_{pivot} and T_{in} . By using transaction references rather than single bits, we can check whether T_{in} or T_{pivot} has committed before T_{out} , and if so, avoid an abort in that case without the possibility of causing a nonserializable execution. In other words, using transaction

references rather than single bits makes the detection of nonserializable executions more precise, with fewer false positives than the basic scheme described in Section 3.1.

Since transactions are only suspended temporarily and then deleted, references between transactions cannot be kept indefinitely. To deal with this, at commit time if either the incoming or outgoing transaction references are set to already-committed transactions, they are replaced with self-references to the committing transaction before it is suspended. In other words, our modified algorithm has the invariant that suspended transactions only ever reference transactions with an equal or later commit. Since we clean up suspended transactions in order of commit, this invariant guarantees that we will never have a reference to a deleted transaction.

3.6 Enhancements and Optimizations

For simplicity, in our description, we did not show all the cases where we could check whether to abort T because both $T.inConflict$ and $T.outConflict$ hold; we have written the check once, in the `commit(T)` operation, and beyond that we only show the extra cases where an abort is done for a transaction that is not the pivot (because the pivot has already committed). In the implementation, we actually abort an active transaction T as soon as any operation of T discovers that both $T.inConflict$ and $T.outConflict$ are true. Likewise, conflicts are not recorded against transactions that have already aborted or that will abort due to both flags being set.

When a conflict between two transactions leads to both conflict flags being set on either one, without loss of correctness either transaction could be aborted in order to break the cycle and ensure serializability. Our prototype implementation follows the algorithm as described above, and prefers to abort the pivot (the transaction with both incoming and outgoing edges) unless the pivot has already committed. If a cycle contains two pivots, whichever is detected first will be aborted. However, for some workloads, it may be preferable to apply some other policy to the selection of which transaction to abort, analogous to deadlock detection policies [Agrawal et al. 1987]. For example, aborting the younger of the two transactions may increase the proportion of complex transactions running to completion. Similarly, policies for victim selection can prioritize particular classes of transactions, reducing the probability of starvation. We intend to explore this idea in future work.

A common pattern in database applications is a sequence of read-modify-write operations. That is, a data item x is read by a transaction T , then some modification to x is performed, and T updates the database with the new value. In the basic algorithm of Section 3.1, this would lead to T holding both an SIREAD lock and an EXCLUSIVE lock on x . Instead, the SIREAD lock can be upgraded to an EXCLUSIVE lock in this case without loss of correctness. Without this optimization, the SIREAD locks would be kept after the transaction commits, leading to unnecessary conflicts and growth of the lock table. For simple transactions, this optimization can avoid keeping transactions suspended if they have no vulnerable reads, and hence no chance of an SI anomaly.

3.7 Mixing Queries at SI with Updates at SSI

It is worth noting that read-only transactions can be run at unmodified SI and mixed in the same system with updates run at Serializable SI. In this configuration, there is no overhead of acquiring SIREAD locks for the queries, and no possibility of queries aborting due to the new unsafe error. The update transactions are serializable when considered separately from the queries, since write skew anomalies are prevented. Thus updates cannot introduce inconsistencies into the data as a result of interleaving and constraints implicit in the application will be maintained.

However, this configuration of transactions taken as a whole is not guaranteed to be serializable. As shown in Fekete et al. [2004], it is possible for a query to read a database state that could not have existed in a serializable execution of the updates. An example from that work is analyzed in Example 2.2.

Regardless, we expect that the possibility of nonserializable queries will be acceptable in many applications and that this configuration will be popular in practice. It may be particularly attractive when making replicated snapshot isolation serializable, so that queries can be satisfied entirely locally at a single replicated node without requiring any communication between nodes.

4. IMPLEMENTATION DETAILS

The algorithm we described was implemented in both Oracle Berkeley DB version 4.6.21 [Olson et al. 1999] and the InnoDB transactional storage plugin version 1.0.1 for MySQL version 5.1.26. This section summarizes the Berkeley DB implementation and describes the InnoDB prototype in detail, explaining how the algorithm was adapted to each engine.

4.1 Berkeley DB

Berkeley DB is an embedded database that supports SI as well as serializable isolation with S2PL. Locking and multiversion concurrency control are performed at the granularity of database pages. This can introduce unnecessary conflicts between concurrent transactions, but means that straightforward read and write locking is sufficient to prevent phantoms.

We added the following to Berkeley DB:

- (1) New error returns `DB_SNAPSHOT_CONFLICT`, to distinguish between deadlocks and update conflicts, and `DB_SNAPSHOT_UNSAFE`, returned according to the algorithm.
- (2) A new lock mode, `SIREAD`, which does not conflict with any other lock modes. The code that avoiding locking for snapshot isolation reads was modified to instead acquire an `SIREAD` lock for Serializable SI transactions.
- (3) Code to clean old `SIREAD` locks from the lock table.
- (4) When an `EXCLUSIVE` lock request for transaction T finds an `SIREAD` lock already held by T , the `SIREAD` lock is upgraded to an `EXCLUSIVE` lock.

Making these changes to Berkeley DB involved only modest changes to the source code. In total, only 692 lines of code (LOC) were modified out of a total

of over 200,000 lines of code in Berkeley DB. Approximately 40% (276 LOC) of the changes related to detecting lock conflicts and a further 17% (119 LOC) related to cleaning obsolete locks from the lock table. Of the code comprising the locking subsystem of Berkeley DB, 3% of the existing code was modified and the total size increased by 10%.

More details are given in Cahill et al. [2008].

4.2 InnoDB Introduction

MySQL is a popular open source relational DBMS [MySQL AB 2006]. One of the architectural features of MySQL is that storage engines are pluggable: new storage engines with different concurrency control or I/O characteristics can be dynamically added to MySQL without changing the query processing front end.

The most stable and popular storage engine for transactional data management in MySQL is InnoDB.¹ Our prototype implementation was created by modifying the InnoDB Plugin for MySQL version 1.0.1 using MySQL version 5.1.26.

InnoDB provides multiversion concurrency control by managing multiple versions of rows, and providing an older version of a row if required to produce a consistent read of a database. If an old version of a row is required, InnoDB uses *undo* records from the log to reconstruct the version that was current when the reading transaction began.

The default isolation level in InnoDB is called REPEATABLE READ, and at this isolation level, ordinary SQL SELECT statements use a consistent read view that does not require locking. The read view is a structure that is allocated when the transaction starts, and includes a list of active transactions (whose changes should not be visible), so that InnoDB can determine which transactions committed before the transaction began. This structure is used by InnoDB to ensure that the correct versions of rows are seen by the transaction.

The REPEATABLE READ isolation level has the interesting property that reads requiring locking, such as reads that are part of an SQL SELECT FOR UPDATE operation, always read the most recently committed values. Subsequent nonlocking reads of the same item will again see the version from the consistent read view (assuming that it was not modified during the transaction). So within a single REPEATABLE READ transaction, the values returned are a mix of snapshot isolation and read committed semantics, depending on whether the operations acquire locks.

InnoDB also provides SERIALIZABLE isolation, which is conceptually equivalent to REPEATABLE READ but with all reads acquiring shared locks. That is, the most recently committed values are always read, and shared locks are held until the transaction commits. Further, InnoDB's SERIALIZABLE isolation level acquires shared locks on the gaps between rows during predicate reads and exclusive locks on the gaps during inserts and deletes in order to prevent phantoms as described in Section 2.4. In summary, InnoDB's SERIALIZABLE isolation is a traditional two-phase locking implementation

¹<http://www.innodb.com>.

that does not take advantage of multiple versions of data items that are available to reads at other isolation levels.

The existing InnoDB semantics are difficult to reason about if updates are performed at any isolation level other than `SERIALIZABLE`. However, the locking implementation of `SERIALIZABLE` isolation in InnoDB negates the concurrency benefits of multiversion concurrency control for reads that form part of an update transaction.

The InnoDB lock manager provides five lock modes: shared, exclusive, intention shared, intention exclusive, and a special lock mode for auto-increment columns. The intention modes are used only at table-level granularity. A set of locks is represented efficiently in the lock manager as a bitmap associated with a page in the buffer manager, indicating which rows on the page are covered by the lock. In this way, locking a row generally only requires a few bits of memory in the lock manager.

Updates to the InnoDB lock table are protected by a global “kernel mutex,” which keeps the code simple but limits scalability on multicore systems.

InnoDB holds a transaction’s locks until the point in transaction commit just before the write-ahead log is flushed. In other words, locks are released and changes become visible before log records are guaranteed to be on stable storage. This early release of locks creates a window of vulnerability in which a query could commit after seeing data values that would be lost after a crash. For short transactions that perform no other I/O, this optimization in InnoDB reduces the duration for which locks are held by 1–2 orders of magnitude. The reasoning given in the InnoDB source code for this design is that as the log is flushed in order, any subsequent update based on the not-yet-flushed data could not be flushed before the data it depends on. However, this reasoning does not apply to queries, which can read data that has not yet been made durable. There are various configuration parameters that can reduce the cost of flushing the log, including group commit and skipping the flush entirely, so releasing locks early is an interesting design choice.

In order to amplify the effects of locking and ensure that data visibility is fully transactional, we changed the order of operations during commit in InnoDB so that locks are not released until after the log has been flushed.

4.3 Adding Snapshot Isolation to InnoDB

As part of our work, we added an implementation of standard Snapshot Isolation (SI) to InnoDB. This involved using the existing `REPEATABLE READ` code to allow SI reads, including those that are part of an update, to use a consistent read view without acquiring shared locks. Records that are updated are locked with exclusive locks, as in `REPEATABLE READ`, and no gap locking is performed.

The first-committer-wins (FCW) rule was implemented by adding a check in the InnoDB code that determines whether old versions are required for a consistent read. If an SI transaction has an exclusive lock on the row and finds that a newer version has been committed since its read view was allocated, a new error `DB.UPDATE_CONFLICT` is returned, indicating that committing the transaction would violate the FCW rule.

Our implementation of SI has the useful property that the read view for an SI transaction is allocated only after any initial lock is acquired for the first statement in the transaction. For example, if the first statement is an SQL UPDATE or SELECT FOR UPDATE, an exclusive lock is acquired before the transaction snapshot for reading is determined. This simple implementation detail can dramatically reduce the number of aborts due to the FCW rule by reordering operations in a transaction or using SELECT FOR UPDATE.

For example, if two instances of a simple transaction that just increments a counter x are started at the same time, our implementation will ensure that one acquires the lock on x and commits before the second one chooses its snapshot. In other words, the second transaction will always read the results of the one that committed first, so there is no possibility of a update after the chosen snapshot, and thus no need to invoke the FCW rule. This issue did not arise in InnoDB previously because the existing REPEATABLE READ isolation level has no FCW rule, and in this simple example, since the data item is locked during the update, the most recent version is used anyway.

4.4 Adding Serializable SI to InnoDB

The first challenge in adding Serializable Snapshot Isolation to InnoDB was representing the SIREAD lock mode. InnoDB already has five lock modes, and packs the 5×5 lock conflict matrix into a 32-bit integer, so that checking for lock compatibility can be performed by bitwise integer operations. Adding a sixth lock mode for SIREADs would have increased the conflict matrix to 6×6 , which cannot fit in the existing representation. Instead, we noted that SIREAD locks are only acquired for rows (or gaps between rows), but never at table granularity. InnoDB includes an “intention shared” lock mode (IS) that is only ever acquired for tables, never rows. The IS locks conflict only with EXCLUSIVE locks, which is exactly what we need for SIREAD locks. Our implementation uses IS locks on rows to represent SIREAD locks in order to avoid adding a new lock mode.

Phantoms are prevented in InnoDB at SERIALIZABLE isolation by gap locking, as described in Section 2.4. We adapted InnoDB’s gap locking to detect and prevent phantoms under Serializable SI, as described in Section 3.4. This was quite straightforward to implement because SERIALIZABLE transactions set a field in the transaction structure indicating that SELECT statements should acquire shared locks for rows and gaps. We use the same field to indicate that SELECT statements in Serializable SI transactions should acquire IS locks instead.

There are two places where read-write conflicts are detected: when reading an old version of a row, and when acquiring locks. In the lock manager, when deciding whether a row lock conflict should cause the transaction to block, we added code to check whether either of the locks involved in the conflict has IS mode. If so, we mark a read-write conflict, where the transaction making the IS lock request is the reader. In the code, to retrieve the version of a row required by a consistent read, we mark a read-write conflict with the owner of any newer version.

In marking a read-write conflict, we first check that the two transactions do, in fact, overlap and that neither is already destined to abort. We then check whether this conflict will cause one of the transactions to abort and if so, only abort that transaction. Last, if neither transaction will abort as a result of the conflict, we mark the incoming and outgoing flags of the two transaction objects as described in Section 3.

When a Serializable SI transaction attempts to commit, we first check whether both incoming and outgoing flags are set, and if so, we roll back the transaction instead and return an error `DB_UNSAFE_TRANSACTION`. We implemented the optimizations described in Section 3.5, where the incoming and outgoing edges are represented by references to two distinct transaction objects, and we roll back only when T_{out} commits before T_{in} .

If any SIREAD locks are held at commit time, the transaction structure is suspended rather than immediately recycled. In the InnoDB implementation, the transaction structure will be reused by the next operation, so a new dummy transaction object is allocated and initialized with the committing transaction's ID, SIREAD locks and conflicts, then the dummy transaction is added to a list of committed Serializable SI transactions that overlap with some running transaction.

If the committing transaction has the oldest read view of all active transactions, the list of committed Serializable SI transactions is scanned and any dummy transactions that no longer overlap with an active transaction are cleaned up and their SIREAD locks released. This eager cleanup of suspended transactions maintains a tight window of active transactions and minimizes the number of additional locks in the lock manager.

The prototype implementation for Berkeley DB, described in Cahill et al. [2008], used a reference count in the transaction object to perform garbage collection when a suspended transaction was no longer overlapping with any running transaction. Our technique has the advantage that aggressive cleanup ensures that only the minimal number of transaction or lock objects will be kept active.

The changes to InnoDB to implement both SI and Serializable SI were modest: around 250 lines of code were modified and 450 lines of code were inserted. There are over 180,000 lines of code in total in InnoDB, so this represents less than 0.4% of the InnoDB source code. More than half of our changes were in the lock manager (modifying 6% of InnoDB's locking code), to allow IS locks on rows and to clean up locks when suspended Serializable SI transactions are freed. The remaining changes were in the InnoDB transaction code and row visibility checking, including code to detect when a read ignores newer versions of a row. In addition, the MySQL front-end query processing code was modified slightly to support setting the new isolation levels and to propagate the new error codes back to clients.

4.5 Generalizing to other Database Engines

A key issue for an implementation is how to keep the SIREAD locks and transaction information after the transaction commits, and then clean up the

transactions and locks as efficiently as possible. In InnoDB, it is reasonable for transactions and locks to remain in the system for some time after commit, and we describe in Section 4.4 the clean up procedure during transaction commit. This is also true for the other open source DBMS engines that we are familiar with, including Berkeley DB and PostgreSQL.

In a different system, where it is not feasible to keep transaction objects suspended after commit, a table would need to be maintained containing the following information: for each transaction ID, the begin and commit timestamps together with an inConflict flag and an outConflict flag. Here is an example.

txnID	beginTime	commitTime	inConf	outConf
100	1000	1100	N	Y
101	1000	1500	N	N
102	1200	N/A	Y	N

Entries in this table can be removed from the table when the commit time of a transaction is earlier than the begin times of all active transactions. In this case, only transaction 102 is still running (since the commit timestamp is not set). The row for transaction 100 can be deleted, since it committed before the only running transaction, 102, began.

Another issue is how to store SIREAD locks if the existing lock manager cannot maintain them, for example if a new lock mode cannot be added or the lock manager cannot be modified to keep SIREAD locks after the owning transaction commits. In this case, a table could be constructed to represent SIREAD locks. Since SIREAD locks never cause blocking, the table would only be used to determine whether a read-write conflict has occurred. Rows in the SIREAD lock table become obsolete and can be deleted when the owning transaction becomes obsolete, that is, when all overlapping transactions have completed.

4.6 Testing

We have performed an exhaustive analysis of the implementation by testing it with all possible interleavings of some set of transactions known to cause write skew anomalies. For example, one test set was as follows:

$$\begin{aligned}
 T_1: & b_1 r_1(x) c_1 \\
 T_2: & b_2 r_2(y) w_2(x) c_2 \\
 T_3: & b_3 w_3(y) c_3.
 \end{aligned}$$

The implementation was tested by writing a program that generated test cases from such a set of transactions, where each test case was a different interleaving of the transactions. Each interleaving was executed by starting a separate client connection for each transaction and synchronizing between the client connections in order to generate the specified interleaving on the server.

During development, this testing regime was invaluable in establishing that conflicts were detected in all relevant code paths through the DBMS. For

example, there are different code paths for queries that involve scans, depending on whether a relevant index is available. By testing different sets of transactions against various configurations of the database, we gained confidence that we had modified InnoDB in all relevant places.

In all cases where the transactions were executed concurrently against the final version of the code, one of the transactions aborted with the new “unsafe” error return. These results were manually checked to verify that no nonserializable executions were permitted although all interleavings committed without error at SI.

5. PERFORMANCE EVALUATION

5.1 Evaluation Setup

The MySQL server was run on a Linux system with an AMD Athlon XP 2600+ CPU with 2GB RAM running openSUSE Linux 11.0 with kernel version 2.6.25, glibc version 2.8, and GCC version 4.3.1. All data was stored using the EXT3 filesystem on a Maxtor 6Y060L0 PATA 7200 RPM hard disk with write cache disabled. MySQL was started with only the following parameters modifying the default configuration (chosen after experimentation to give best throughput on our server without sacrificing ACID properties):

```
--max-connections=300 --table-cache=1000
--innodb_buffer_pool_size=1G --innodb_log_file_size=256M
--innodb_log_buffer_size=16M --innodb_flush_method=fsync.
```

The benchmark clients were run on a FreeBSD 7.0 system with an Intel Core(TM)2 Quad Q9300 at 2.50GHz and 4GB RAM using the Java HotSpot(TM) 64-Bit Server VM (build 1.6.0.03-p4) with the MySQL connector JDBC driver version 5.1.6. The systems were connected via a 100MBit Ethernet switch and were otherwise idle while the experiments ran.

We deliberately ran the database server on a slower machine so that it would reach CPU, memory, and I/O saturation under different conditions during testing.

Each configuration was measured between three and five times, and the database cache was prewarmed by loading all of the data into the database before measuring. Tests were run for one minute without measuring to allow the workload to reach a steady state, and results were measured for a further two minutes. All graphs include 95% confidence intervals (assuming a normal distribution of results), although in many cases the confidence intervals are too small to be visible.

5.2 `sibench`: A Microbenchmark for Snapshot Isolation

To explore the performance impact of avoiding blocking for read-write conflicts, we first illustrate the trade-off between bottlenecks including CPU time, lock contention and write-ahead log flushes during commits.

We created a very simple synthetic microbenchmark called `sibench` based on a read-write conflict between an query transaction and an update transaction.

The benchmark uses a single table called `sibench` with two non-null integer columns: `id`, which is a primary key, and `value`. Since there is only a single edge in the static dependency graph, there is no possibility of deadlocks or write skew, so no transactions are required to roll back during the execution of the benchmark. We verified during the evaluation, that no transactions deadlocked or experienced write skew.

The query transaction returns the `id` with the smallest value in the table. This transaction was designed so that the DBMS must scan all rows in the table, and so that there is some CPU cost associated with processing the rows, but the final result is small, minimizing communication cost. The query transaction contains a single line of SQL:

```
SELECT id FROM sitest ORDER BY value ASC LIMIT 1.
```

The update transaction simply increments the value of a row chosen from a uniform random distribution of the identifiers. It is also a single line of SQL:

```
UPDATE sitest SET value = value + 1 WHERE id = :id.
```

Note that a consequence of the optimization described in Section 4.3, where the read snapshot is not determined until after any lock required for the first statement has been acquired, conflicts between concurrent updates in `sibench` will result in blocking but not aborts.

We concentrate on the primary parameter to `sibench`: I , the number of rows in the table. The value of I controls the number of rows read by the query, and hence the number of lock manager requests when the query runs at S2PL or Serializable SI. However, the query also involves a sort, so increasing I also increases the amount of CPU required to execute the benchmark. Further, for small values of I , the rate of write-write conflicts between simultaneous updates is high, so increasing I reduces the proportion of write-write versus read-write conflicts.

In all of the results presented here, each client thread performs an equal number of query and update transactions.

We explore cases where $I \in \{10, 100, 1000\}$. When $I = 10$ and the number of query and update transactions are equal, there are 10 times more rows read than updated. In the extreme case where $I = 1000$, there are 1000 rows read for each row updated.

Figure 11 shows that for workloads with low to moderate ratios of reads to updates, SI scales to more than double the throughput of S2PL, since SI eliminates blocking for read-write conflicts. With such a small set of rows, the rate of conflicts between concurrent updates is high, particularly when $MPL \geq 10$. There is virtually no overhead for Serializable SI compared with SI.

Figure 12 shows the results for a workload with a high rate of reads per update (100 times more reads than updates). In this scenario, SI scales to more than triple the throughput of S2PL, and with the larger set of rows, the rate of conflicts between concurrent updates is moderate, so that overall throughput is higher than when $I = 10$. There is virtually no overhead for Serializable SI compared with SI until $MPL 50$, at which point the CPU becomes 100% utilized at Serializable SI, but the CPU is not saturated at SI.

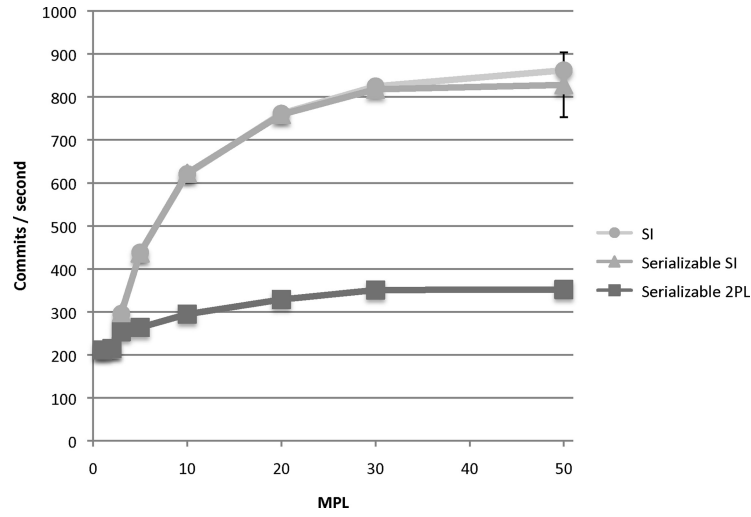


Fig. 11. Throughput of sibench with 10 items at SI, S2PL and Serializable SI.

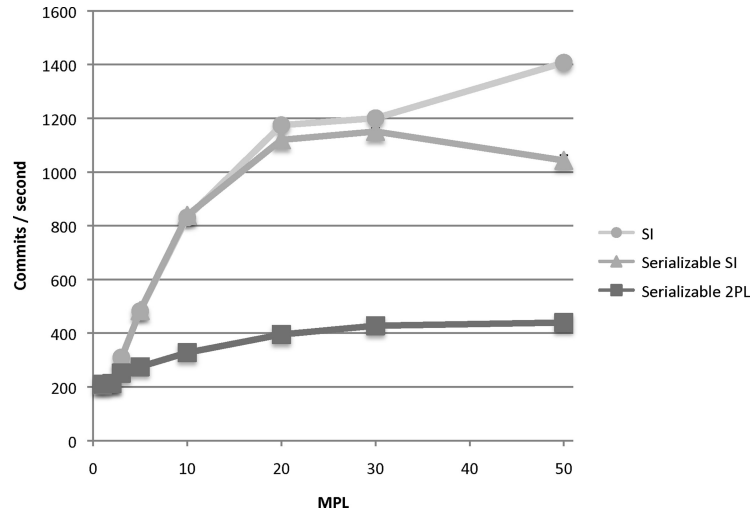


Fig. 12. Throughput of sibench with 100 items at SI, S2PL and Serializable SI.

When I is increased further, to 1000, in Figure 13, more CPU time is required to sort the rows during each query and there are 10 times more requests to the lock manager than in Figure 12. Here we see that the gap between SI and Serializable SI is more pronounced, for two reasons:

- the CPU becomes saturated earlier under Serializable SI, at around MPL 20 rather than at MPL 30 under SI, due to the additional work involved in managing SIREAD locks in the lock table; and
- there is now significant contention for the mutex protecting the lock table. As mentioned in Section 4.2, the InnoDB lock table is protected by a single, global

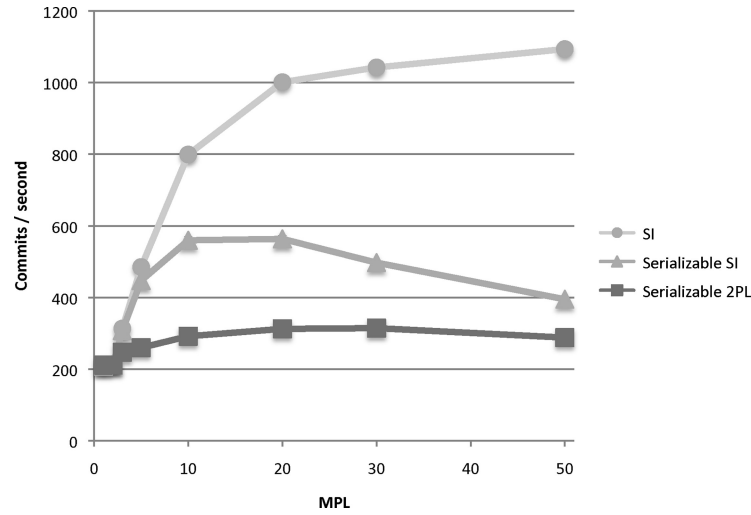


Fig. 13. Throughput of sibenx with 1000 items at SI, S2PL and Serializable SI.

mutex, so concurrent lock requests contend for this mutex even when there is no chance that the requested locks would conflict. A more sophisticated lock manager should scale better under this Serializable SI workload, whereas the S2PL throughput is constrained by blocking caused by logical lock conflicts rather than a limitation of the lock manager implementation.

5.3 The TPC-C++ Benchmark

When proposing changes to the core of a database system, it is natural to attempt an evaluation using one of the standard benchmarks of the Transaction Processing Council, such as the TPC-C benchmark [Transaction Processing Performance Council 2005]. TPC-C models the database requirements of a business selling items from a number of warehouses to customers in some geographic districts associated with each warehouse.

It has long been known that TPC-C is serializable when run at snapshot isolation, and this was formally proved in Fekete et al. [2005]. We adopt their abbreviations and analysis, and Figure 15 gives the static dependency graph they derived for TPC-C.

In order to evaluate the effectiveness and overhead of avoiding SI anomalies, we need a benchmark that can experience anomalies when run at SI. Here we describe a modification to the standard TPC-C that we call TPC-C++.

The schema and cardinality of TPC-C are given in Figure 14, and the benchmark consists of the following transactions:

- approximately 43% **New Order** transactions, abbreviated as NEWO, where a customer places an order for a list of different items;
- at least 43% **Payment** transactions, abbreviated as PAY, where a customer pays for an order placed earlier;

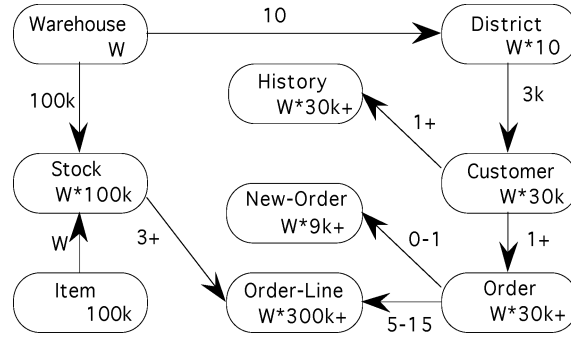
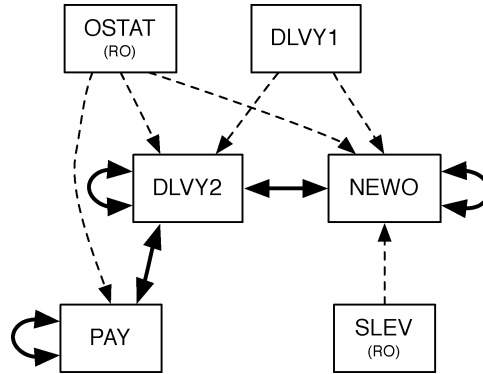


Fig. 14. The tables and cardinality of TPC-C.


 Fig. 15. Static dependency graph for standard TPC-C, with *rw*-conflicts shown as dashed edges and *ww*-conflicts shown in bold [Fekete et al. 2005].

- 4% **Delivery** transactions, abbreviated as DLVY, which marks an order in each of 10 districts delivered. Following Fekete et al. [2005], we split the Delivery transaction into two parts: DLVY1, where there are no new orders, so there is no work for the transaction to do; and DLVY2, where there is at least one new order awaiting delivery;
- 4% **Order Status** transactions, abbreviated as OSTAT, where a customer checks the status of an order; and
- 4% **Stock Level** transactions, abbreviated as SLEVEL, where the company checks whether a warehouse is running low on any recently-ordered items.

Our use of TPC-C as a basis for the evaluation is intended only to provide a widely-understood database workload. We do not attempt to get certifiable TPC-C results. In particular, we:

- ignore terminal emulation entirely: only the database transactions are considered;
- leave out waits between transactions to increase the load on the database for a given number of clients;

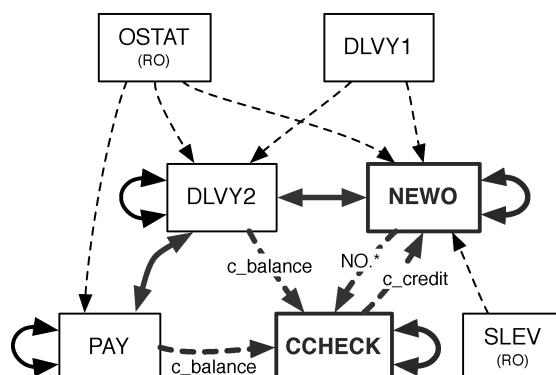


Fig. 16. Static dependency graph for TPC-C++.

- report total transaction throughput rather than only the number of New Order transactions that are included in tpmC;
- avoid unnecessary conflicts without the complexity of partitioning the tables by assuming that the constant `w_tax` field in the Warehouse table can be cached by clients.

TPC-C++ keeps the same schema as TPC-C but adds a new transaction type called *Credit Check*. The purpose of this transaction is to check a customer’s total outstanding balance and update their credit status if the customer exceeds their credit limit. Such a transaction could execute either as part of a batch job, updating the status for all customers meeting some criteria, or as part of a workflow during an interaction between the customer and the company.

A customer’s total outstanding balance combines both the delivered, unpaid orders (tracked in the `c_balance` field of the Customer table) and total value of new orders that are not yet delivered. As shown in Figure 16, although simple, the Credit Check transaction significantly complicates the static dependency graph. A Credit Check transaction reads the NewOrder table, into which the New Order transaction inserts, and also reads the `c_balance` field of a customer record, which is updated by both the Delivery and Payment transactions. It updates the customer’s `c_credit` field, which is read by the New Order transaction. If two Credit Check transactions were run concurrently for the same customer, each would attempt to update the same customer record, hence the *ww*-conflict with itself.

Sample code for the Credit Check transaction is given in Figure 17 in the style of appendix A of the TPC-C specification. We execute Credit Check transactions with the same probability as Delivery transactions (a minimum 4% of the mix) unless otherwise indicated.

In a DBMS that performs locking and versioning at row-level granularity with the standard physical layout of the Customer table, the conflicts between the Credit Check transaction and either Delivery or Payment would in fact be write-write conflicts, even though the transactions update different fields in the Customer table. It is common practice in such situations to partition the table in order to improve concurrency, and the TPC-C specification explicitly

```

EXEC SQL SELECT c_balance, c_credit_lim INTO :c_balance, :c_credit_lim
FROM Customer
WHERE c_id = :c_id AND c_d_id = :d_id AND c_w_id = :w_id

EXEC SQL SELECT SUM(ol_amount) INTO :neworder_balance
FROM OrderLine, Orders, NewOrder
WHERE ol_o_id = o_id AND ol_d_id = :d_id AND ol_w_id = :w_id AND
      o_d_id = :d_id AND o_w_id = :w_id AND o_c_id = :c_id AND
      no_o_id = o_id AND no_d_id = :d_id AND no_w_id = :w_id

if (c_balance + neworder_balance > c_credit_lim)
    c_credit = "BC";
else
    c_credit = "GC";

EXEC SQL UPDATE Customer SET c_credit = :c_credit
WHERE c_id = :c_id AND c_d_id = :d_id and c_w_id = :w_id

EXEC SQL COMMIT WORK;
    
```

Fig. 17. Sample code for the Credit Check Transaction as SQL embedded in C.

permits partitioning and discusses some possible partitioning schemes for the Customer table. If `c_balance` and `c_credit` were stored in different partitions, the conflicts would be as shown even in a DBMS with row-level locking and versioning.

In the modified SDG of Figure 16, there are two pivots: New Order and Credit Check. There are several cycles: the simplest is from Credit Check to New Order and back again. This simple cycle corresponds to a credit check that runs concurrently with the placement of a new order. As a result, the Credit Check transaction does not see the effect of the New Order, and the New Order transaction does not see the result of the Credit Check. This is a straightforward write skew.

More complex cycles are also possible. For example, the cycle from Credit Check through New Order, Delivery, and Payment, back to Credit Check. That cycle can give rise to the following example of a nonserializable execution.

Example 5.1. Assume that the customer is informed during a New Order transaction if they have a bad credit rating. The status is displayed on the terminal, so an operator might indicate to the customer that the order will be delayed or that additional charges will apply. Further, assume that some customer has a credit limit of \$1000 and that `c_balance` is initially \$900.

Customer		System
Operation	Unpaid total	
New Order (\$200)	\$1100	begin Credit Check calculate total balance = \$1100 ⇒ <code>c_credit = "BC"</code> commit
Payment (\$500)	\$600	
New Order (\$100)	\$700	
New Order (\$150)	\$850 + bad credit	

The customer's first New Order transaction exceeds their credit limit. The next transaction is a Payment, which reduces the unpaid total back under the limit. If the next order were marked as "bad credit," the customer would recognize that their overdraft had been detected before the payment was processed.

However, in this execution, a Credit Check transaction begins concurrently with the Payment transaction and commits concurrently with the following New Order transaction. If the credit check runs at Snapshot Isolation, it will calculate an unpaid total of \$1100, as it does not see any changes from either of the concurrent transactions. It therefore commits a "bad credit" status, but the customer does not see the result until *after* successfully placing an order with a "good credit" status.

There is no way that this could have happened in a serializable execution. If the Credit Check had been serialized before the Payment, the \$100 order would have been marked as bad credit. If the Credit Check were serialized after the payment, or after the \$100 order, it would have calculated an unpaid balance below the \$1000 credit limit and would have set `c_credit = "GC"`.

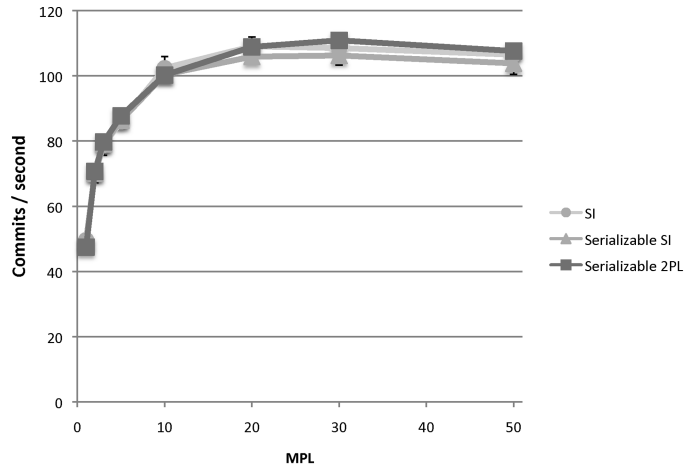
Figure 18 shows the throughput and error rates for the standard TPC-C mix of transactions plus the Credit Check transaction running at approximately 4% of the mix. The scaling factor is $W = 10$, which results in approximately 1.2 GB of data—more than will fit into the 1GB cache, so some I/O is required to satisfy queries.

In Figure 18(a), we see that the throughputs for all three isolation levels are virtually identical. This was by far the most common case when we ran TPC-C++ with a variety of parameters. In other words, concurrency control, and specifically blocking on locks, is not the primary bottleneck in TPC-C, or our variant, TPC-C++. The TPC-C schema is designed so that conflicts become rarer as the data volume is scaled, so it is not surprising to see that the choice of isolation level is not the main contributor to performance.

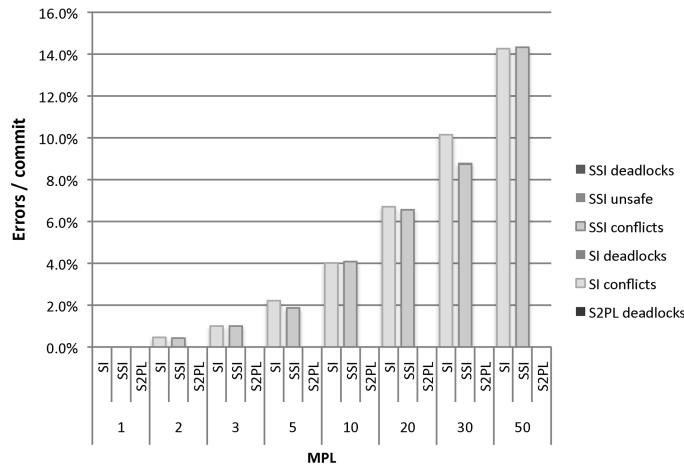
However, recall that when TPC-C++ is run with ordinary SI, the execution is not serializable in general. Undeclared consistency constraints may be violated. In Figure 18(b) we give a breakdown of error rates in a stacked bar graph. Here the situation is simple because there are no deadlocks at any isolation level and the FCW is the overwhelming cause of transaction aborts. "Unsafe" errors contribute only a tiny fraction of the total (less than 1% of the FCW errors). Figure 18 is the common case where avoiding SI anomalies has no measurable impact on performance.

We noticed the moderate to high rate of SI conflicts due to the first-committer-wins rule in 18(b) and wanted to understand what was causing the conflicts. The benchmark is run without partitioning, so updates to the field `d_next_o_id` for a district in the New Order transaction conflict unnecessarily with updates to the `d_ytd` field in the Payment transaction.

To investigate whether this unnecessary conflict is the source of the higher than expected error rates, we ran the benchmark again but modified the Payment transaction to avoid updating the `d_ytd` field. The results are shown in Figure 19. Comparing Figure 18(a) with Figure 19(b), we can see that the rate



(a) Throughput of TPC-C++ at SI, S2PL and Serializable SI.

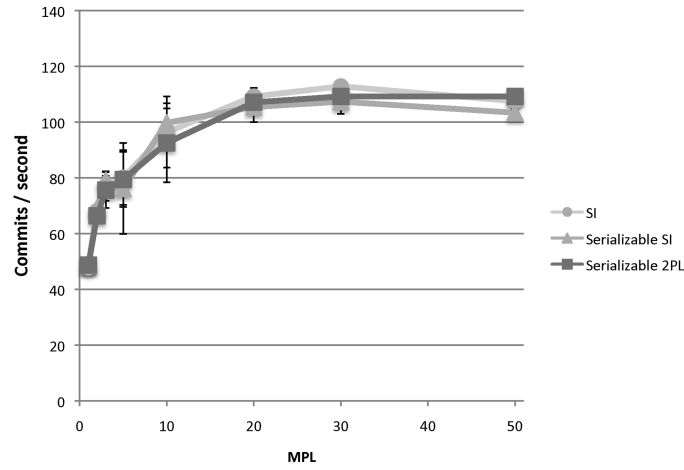


(b) Error rates at SI, S2PL and Serializable SI.

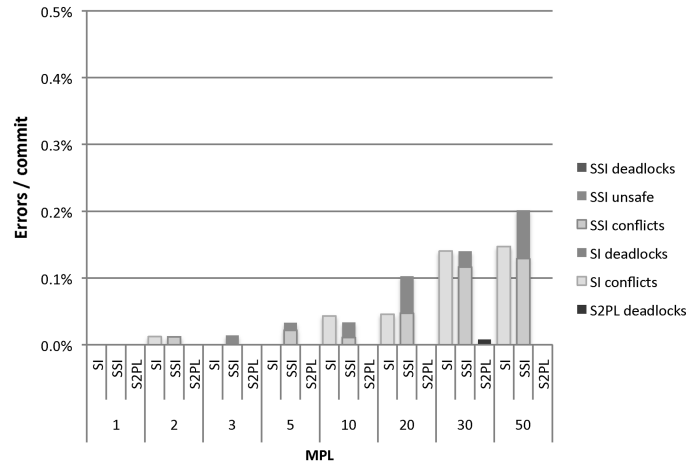
Fig. 18. Throughput and error rates for TPC-C++ with 10 warehouses.

of update conflicts at SI and Serializable SI has been reduced by approximately a factor of 100. This indicates that those fields contribute the overwhelming majority of update conflicts in TPC-C running at SI, and would not occur if the District table were partitioned such that the `d_next_o_id` and `d_ytd` fields were stored in separate partitions.

Next we tried to configure TPC-C++ to highlight the effects of concurrency control. Multiversion concurrency control provides most benefit in situations where read-write conflicts are the primary bottleneck, which corresponds to the dashed lines in the static dependency graph in Figure 16. Here we focus on the edge between the New Order transaction and the Stock Level transaction.



(a) Throughput of TPC-C++ at SI, S2PL, and Serializable SI.

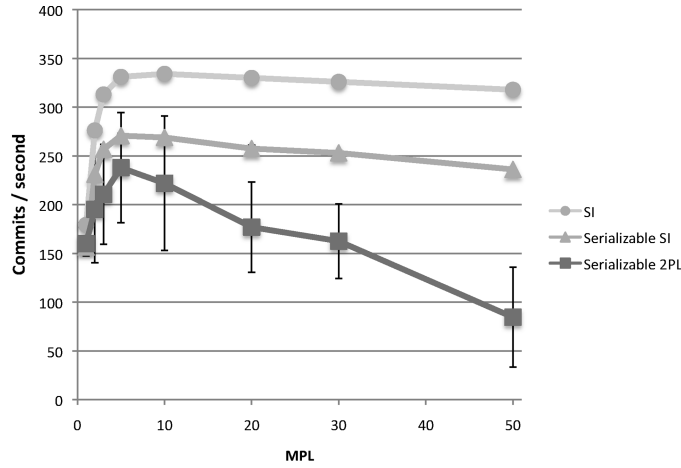


(b) Error rates at SI, S2PL, and Serializable SI.

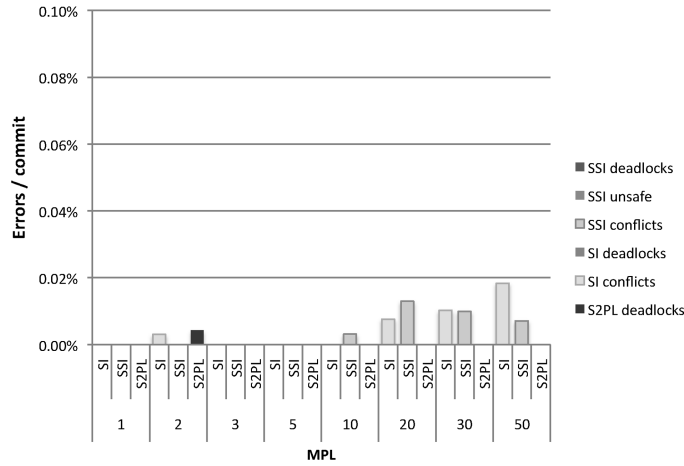
Fig. 19. Throughput and error rates for TPC-C++ with 10 warehouses, skipping year-to-date updates.

Figure 20 shows the results for running a mix of just the New Order and Stock Level transactions, where 10 Stock Level transactions are executed for each New Order transaction. The Stock Level transaction examines the 20 most recent orders, with an average of 10 order lines per order. The New Order transaction modifies approximately 20 rows on average. The result is that this mix is skewed, with approximately 100 rows read for each row updated. We call this the *Stock Level Mix*.

In Figure 20(a), we see that Serializable SI performs approximately 20% more slowly than SI. However, since the data for this configuration is larger than



(a) Throughput of TPC-C++ W=10 Stock Level Mix at SI, S2PL and Serializable SI.

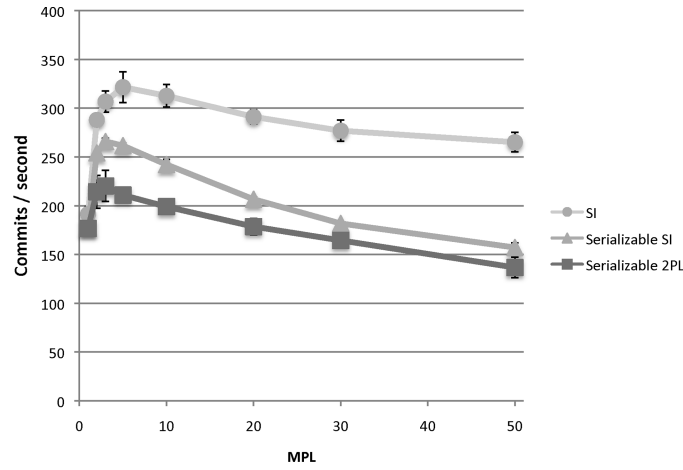


(b) Error rates of TPC-C++ W=10 Stock Level Mix at SI, S2PL and Serializable SI.

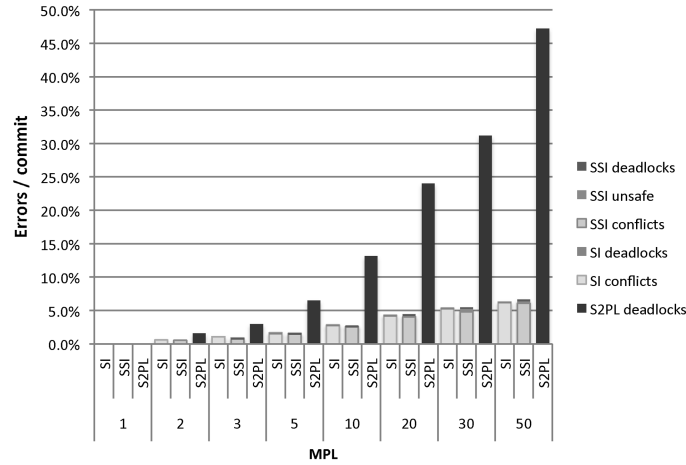
Fig. 20. Throughput and error rates for TPC-C++ Stock Level Mix, W=10.

the cache, and S2PL holds shared locks that block updates while performing I/O, we also see that the performance of S2PL is far worse than Serializable SI, with negative scaling after MPL=5 and much higher variability.

Finally, we present Figure 21(b), where the data volume is scaled to much smaller than normal in order to make conflicts more common. These results were obtained with 1 warehouse, 10 districts, 100 customers per district, and 1000 items. The same mix of 10 Stock Level transactions per New Order transaction was measured, and Figure 21(a) shows that again, Snapshot Isolation performs best, with Serializable SI between 10 and 20% faster than S2PL. The rate of deadlocks at S2PL in this configuration may be prohibitive.



(a) Throughput of TPC-C++ W=1, tiny data, Stock Level Mix at SI, S2PL, and Serializable SI.



(b) Error rates of TPC-C++ W=1, tiny data, Stock Level Mix at SI, S2PL, and Serializable SI.

Fig. 21. Throughput and error rates for TPC-C++ Stock Level Mix, W=1, tiny data.

Summarizing our observations:

- If read-write conflicts are the primary bottleneck, using SI can significantly out-perform S2PL.
- Read-write locking is generally not the performance bottleneck for TPC-C++.
- Serializable SI gives performance close to SI unless the database is spending 100% CPU servicing reads: in that case the lock manager becomes the bottleneck in InnoDB. A more sophisticated lock manager that does not rely on a single, global mutex might help somewhat, but some of the conflicts are inherent in the data access patterns.

- If we change the transaction mix to have many read-write conflicts, we see that SI outperforms S2PL and Serializable SI is somewhere in between. For data sizes larger than the cache size, such as $W=10$, the cost of holding a shared lock while waiting for I/O becomes very expensive for S2PL. Neither SI or Serializable SI suffer from this.
- In none of these cases were unsafe errors a significant contributor to the performance of the benchmark. This is because the true rate of nonserializable interleavings is small and the rate of false positives is also low. Even when skewing the transaction mix to induce more read-write conflicts, the pattern required for write skew was quite rare in practice.
- When implemented at page-level granularity (in Oracle Berkeley DB), there was a moderate rate of false positives, where page-level conflicts between transactions operating on different records caused unnecessary unsafe aborts. With row-level granularity (in InnoDB) together with the optimizations in Section 3.5, the rate of false positives is much lower.

To measure any difference between the throughput with SI and Serializable SI, we observed that the following combination of conditions is required.

- (1) The workload is dominated by queries run at Serializable SI (so Serializable SI is using the lock manager but SI is not).
- (2) The DBMS is using 100% CPU (so Serializable SI is frequently interrupted while holding the lock manager mutex).
- (3) There are many simultaneous threads in the DBMS (so the number of simultaneous locks gets large and the queue waiting for the lock manager mutex gets long).
- (4) There is no think time in the benchmark.

Each of these conditions can be addressed in turn as follows.

- (1) Run pure queries at SI. As noted in Fekete et al. [2004], this can lead to anomalies where a read-only query sees a database state that could not have occurred in a serializable execution. However, data consistency will be maintained, because updates will be serializable, as described in Section 3.7.
- (2) Buy a faster CPU: we used a single-CPU machine to generate results because it was more difficult to measure differences between the isolation levels on a faster quad-CPU machine.
- (3) Limit the number of threads in the DBMS—we overrode the InnoDB default of a maximum of eight simultaneous threads to amplify the trends.
- (4) Use a benchmark that reflects a real application—we deliberately left the TPC-C think time out of our benchmark because we wanted to stress the DBMS.

6. CONCLUSIONS AND FUTURE WORK

This article presents a new method for implementing serializable isolation based on a modification of snapshot isolation. Prototype implementations of

the algorithm in both Oracle Berkeley DB and the InnoDB transactional storage backend for MySQL are described and shown to perform significantly better than two-phase locking in a wide variety of cases, usually comparably with snapshot isolation.

One property of Berkeley DB that simplified that prototype implementation was working with page level locking and versioning. This article extends the basic algorithm to support locking and versioning with row-level granularity, and uses a gap locking protocol to detect and prevent phantoms. The enhanced algorithm was the basis of the InnoDB prototype, and our evaluation shows the advantages of finer granularity in much lower rates of false positives compared with the Berkeley DB prototype.

The Serializable SI algorithm is conservative, and in some cases leads to slightly higher abort rates than SI. However, the impact on system throughput is generally small and in most cases when read-write conflicts are the primary bottleneck, performance is substantially better than the locking serializable isolation built into InnoDB.

Our evaluation highlighted the importance of a scalable lock manager. In the future we hope to implement this algorithm in a system with a highly concurrent lock manager to further explore its performance. In addition, we are investigating how to adapt this technique to a distributed setting to provide serializable isolation for replicated database systems.

REFERENCES

- ADYA, A. 1999. Weak consistency: A generalized theory and optimistic implementations for distributed transactions. Ph.D. thesis, Laboratory for Computer Science, Massachusetts Institute of Technology.
- AGRAWAL, R., CAREY, M. J., AND McVOY, L. 1987. The performance of alternative strategies for dealing with deadlocks in database management systems. *IEEE Trans. Softw. Eng.* 13, 12, 1348–1363.
- ALOMARI, M., CAHILL, M. J., FEKETE, A., AND RÖHM, U. 2008. The cost of serializability on platforms that use snapshot isolation. In *Proceedings of the 24th International Conference on Data Engineering (ICDE)*. 576–585.
- BERENSON, H., BERNSTEIN, P., GRAY, J., MELTON, J., O’NEIL, E., , AND O’NEIL, P. 1995. A critique of ANSI SQL isolation levels. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. ACM Press, 1–10.
- BERNSTEIN, A., LEWIS, P., AND LU, S. 2000. Semantic conditions for correctness at different isolation levels. In *Proceedings of the 16th International Conference on Data Engineering (ICDE)*. IEEE, 57–66.
- BERNSTEIN, P. A. AND GOODMAN, N. 1983. Multiversion concurrency control—theory and algorithms. *ACM Trans. Datab. Syst. (TODS)* 8, 4, 465–483.
- BOBER, P. M. AND CAREY, M. J. 1992. On mixing queries and transactions via multiversion locking. In *Proceedings of the Eighth International Conference on Data Engineering (ICDE)*. IEEE Computer Society, Washington, DC, 535–545.
- CAHILL, M. J., RÖHM, U., AND FEKETE, A. D. 2008. Serializable isolation for snapshot databases. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, 729–738.
- CAREY, M. J. AND MUHANNA, W. A. 1986. The performance of multiversion concurrency control algorithms. *ACM Trans. Comput. Syst.* 4, 4, 338–378.
- CHAN, A., DAYAL, U., FOX, S., GOODMAN, N., RIES, D. R., AND SKEEN, D. 1983. Overview of an ADA compatible distributed database manager. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*. ACM, New York, NY, 228–237.

- ESWARAN, K. P., GRAY, J., LORIE, R. A., AND TRAIGER, I. L. 1976. The notions of consistency and predicate locks in a database system. *Comm. ACM* 19, 11, 624–633.
- FEKETE, A. 1999. Serializability and snapshot isolation. In *Proceedings of the Australian Database Conference*. Australian Computer Society, 201–210.
- FEKETE, A. 2005. Allocating isolation levels to transactions. In *Proceedings of the 24th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*. ACM, New York, NY, 206–215.
- FEKETE, A., LIAROKAPIS, D., O'NEIL, E., O'NEIL, P., AND SHASHA, D. 2005. Making snapshot isolation serializable. *ACM Trans. Datab. Syst.* 30, 2, 492–528.
- FEKETE, A., O'NEIL, E., AND O'NEIL, P. 2004. A read-only transaction anomaly under snapshot isolation. *ACM SIGMOD Record* 33, 3, 12–14.
- GRAY, J. AND REUTER, A. 1993. *Transaction Processing: Concepts and Techniques*. Morgan Kaufmann.
- HADZILACOS, T. 1988. Serialization graph algorithms for multiversion concurrency control. In *Proceedings of the 7th ACM SIGACT-SIGMOD-SIGART Symposium on Principles of Database Systems (PODS)*. ACM, New York, NY, 135–141.
- JACOBS, K., BAMFORD, R., DOHERTY, G., HAAS, K., HOLT, M., PUTZOLU, F., AND QUIGLEY, B. 1995. Concurrency control, transaction isolation and serializability in SQL92 and Oracle7. Oracle White Paper, Part No A33745.
- JORWEKAR, S., FEKETE, A., RAMAMRITHAM, K., AND SUDARSHAN, S. 2007. Automating the detection of snapshot isolation anomalies. In *Proceedings of the 33rd International Conference on Very Large Data Bases (VLDB)*. VLDB Endowment, 1263–1274.
- KUNG, H. T. AND ROBINSON, J. T. 1981. On optimistic methods for concurrency control. *ACM Trans. Datab. Syst.* 6, 2, 213–226.
- LOMET, D. B. 1993. Key range locking strategies for improved concurrency. In *Proceedings of the 19th International Conference on Very Large Data Bases (VLDB)*. Morgan Kaufmann Publishers Inc., San Francisco, CA, 655–664.
- MOHAN, C. 1990. ARIES/KVL: a key-value locking method for concurrency control of multi-action transactions operating on b-tree indexes. In *Proceedings of the 16th International Conference on Very Large Databases (VLDB)*. Morgan Kaufmann Publishers Inc., 392–405.
- MOHAN, C. AND LEVINE, F. 1992. ARIES/IM: an efficient and high concurrency index management method using write-ahead logging. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, 371–380.
- MOHAN, C., PIRAHESH, H., AND LORIE, R. 1992. Efficient and flexible methods for transient versioning of records to avoid locking by read-only transactions. In *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. ACM, New York, NY, 124–133.
- MYSQL AB. 2006. *MySQL Administrator's Guide and Language Reference 2nd Ed.* MySQL Press.
- OLSON, M. A., BOSTIC, K., AND SELTZER, M. I. 1999. Berkeley DB. In *Proceedings of the USENIX Annual Technical Conference, FREENIX Track*. 183–191.
- RAZ, Y. 1993. Commitment ordering-based distributed concurrency control for bridging single and multi version resources. In *Proceedings of the 3rd International Workshop on Research Issues in Data Engineering: Interoperability in Multidatabase Systems (RIDE-IMS)*. IEEE, 189–198.
- REED, D. P. 1978. Naming and synchronization in a decentralized computer system. Tech. rep., MIT, Cambridge, MA.
- REED, D. P. 1983. Implementing atomic actions on decentralized data. *ACM Trans. Comput. Syst.* 1, 1, 3–23.
- SARIN, S. 2009. Dynamic transaction serializability in netezza performance server. *New England Database Day*, <http://db.csail.mit.edu/nedbdays09/htdocs/papers.php>.
- SHI, V. T.-S. AND PERRIZO, W. 2002. A new method for concurrency control in centralized database systems. In *Computers and Their Applications*, R. E. Gantenbein and S. Y. Shin, Eds. ISCA, 184–187.
- TADA, H., HIGUCHI, M., AND FUJII, M. 1997. A concurrency control algorithm using serialization graph testing with write deferring. *Trans. Inform. Proc. Soc. Japan* 38, 10 (Oct.).

- TRANSACTION PROCESSING PERFORMANCE COUNCIL. 2005. TPC-C benchmark specification. <http://www.tpc.org/tpcc>.
- WEIHL, W. E. 1987. Distributed version management for read-only actions. *IEEE Trans. Softw. Eng.* 13, 1, 55–64.
- WEIKUM, G. AND VOSSEN, G. 2002. *Transactional Information Systems: Theory, Algorithms and the Practice of Concurrency Control and Recovery*. Morgan Kaufmann.
- YANG, Y. 2007. The adaptive serializable snapshot isolation protocol for managing database transactions. M.S. thesis, University of Wollongong, NSW Australia.

Received October 2008; revised February 2009, July 2009; accepted July 2009